

Server-Factory-Additional-Components

The supporting cast of the Server Factory provisioning toolchain.

Summary

Beyond Mail Server Factory and the Core Framework, the Server-Factory org contains several smaller components: per-service “factories” (Web Service, SonarQube, Caching Proxy), declarative configuration packs (Docker/Stack/Software Definitions), and shared Utils. This consolidated page covers them honestly — several are early-stage or stub-documented — rather than as fully specified products.

Short description

A group of Server Factory supporting repos: Web-Service-Factory, SonarQube-Factory, and Caching-Proxy-Factory (per-service provisioning tools, mostly early-stage); Docker/Stack/Software-Definitions (declarative config packs consumed by the framework); and Utils (SSH-access helpers and general tooling). All build on Core Framework.

Long description

This page consolidates the remaining Server-Factory repositories because, individually, most are small or intentionally under-documented, and briefing each as a finished product would overstate their maturity. They fall into three groups. **Service factories** follow the Mail Server Factory pattern for other server roles: **Caching-Proxy-Factory** (“Run your own caching proxy server”) lists caching proxy, self-signed certificate, and a security-certificate-obtain HTTP endpoint as key features; **SonarQube-Factory** (“Run your own SonarQube server”) is aimed at software-development use; and **Web-Service-Factory** instantiates and configures a web server to deploy targets such as websites and

micro-services. All three are Kotlin projects built on the Core Framework, but their public READMEs are largely placeholders (“Tbd.” for compatibility, specifications, setup, and usage) — so their concrete capabilities beyond the stated intent are UNVERIFIED. **Definitions packs** — **Docker-Definitions**, **Stack-Definitions**, and **Software-Definitions** — are declarative configuration repositories that the framework consumes to know how to build/deploy Docker images, stacks, and software; they are version-pinned data packs rather than applications. **Utils** provides general helpers for the family, including an `init_ssh_access.sh` script that generates an SSH key and installs it on a remote host to enable passwordless root access for subsequent provisioning. Together these components round out the provisioning toolchain around the flagship Mail Server Factory.

Why we built it

The Server Factory model is built to generalize: once you can provision a mail server from a declarative description, the same engine should be able to provision web servers, caching proxies, and code-quality servers too — fed by reusable definition packs and shared utilities rather than bespoke logic per role. These repos are that generalization in progress, extending the proven pattern to new server types. Their value here is as evidence of the model’s reach; their maturity varies, and this page is deliberately clear about which are direction and which are done.

Why it’s a game-changer (measured)

As a set, they demonstrate the Core Framework’s reusability across server types and separate declarative data (Definitions) from execution (factories). Individually, the service factories are early-stage and should be presented as direction, not finished products.

What’s innovative

- One provisioning framework generalized across mail/web/caching-proxy/SonarQube roles.
- Declarative Definitions packs (Docker/Stack/Software) decoupled from the execution engine.
- Shared Utils (e.g. one-command passwordless SSH bootstrap) reused across factories.

Challenges & solutions

- **Reusing one engine across server roles:** addressed by building each factory on Core Framework.
- **Separating configuration from code:** addressed by the Definitions repos as version-pinned data packs.
- **(UNVERIFIED):** the service factories' READMEs are placeholders; their implementation completeness is not verifiable from public docs — present as early-stage.

Tech stack (why + how)

- **Kotlin** — Web-Service-Factory, SonarQube-Factory, Caching-Proxy-Factory (built on Core Framework).
- **Shell** — Utils and the Definitions packs (scripts/config).
- **Gradle** — `./gradlew test build/test` flow across the factories.
- **Docker** — target runtime described by Docker-Definitions.
- **SSH / OpenSSH** — Utils' passwordless-access bootstrap.
- **SonarQube** — the server SonarQube-Factory provisions (and which Mail Server Factory reports a clean gate against).

Honesty note: most of these repos are forks within the org; the service factories are placeholder-documented and marked UNVERIFIED per constitution §11.4.6. Ranked clearly below Mail Server Factory and Core Framework.